Call Stacks: No More Free Passes For Malware

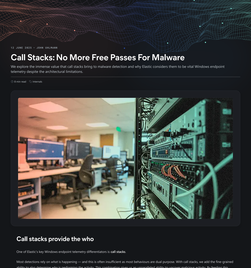
Call stacks are a powerful tool for identifying in-memory tradecraft and rifle hacks commonly used by threat actors. With call stacks, we add the finer-grained ability to also determine who is performing the activity. This combination gives us an unexploitable edge in attempts to uncover malicious activity. By feeding this data into machine learning **Stacks Detect** can track and signal, we can quickly respond to emerging threats.

Call stacks are a beautiful lie

In computer science a **stack** is said to be, first-out data structure. Similar to a stack of physical items, it is only possible to add or remove the top-element. A **call stack** is a stack that contains information about the currently active subroutine calls.

On a Windows, this call stack is managed by operating system execution functions in the DLL kernelbase on the CPU, such as **VirtualAlloc**, Intel MT, Intel MPX, Intel **BTS**, and **Anti Architecture UM**. These tracking features were designed for performance profiling and debugging purposes, but can be used in some security scenarios as well. Modern attack techniques generally avoid the use of Windows API functions, but have to ultimately pass through a common call chain mechanism called **stack walking**.

Ret-addr protections the most common regime[1] [2] [3] is precisely/only points to a stack check, and there are effectively instructions to read and write the data on this stack. Additionally, the `call` instruction transfers control to a new subroutine but also saves a return address at the memory address referenced by the stack pointer. `ret` instructions then read the address stored at that instruction pointer to return to where it left off. Functions in most programming languages are typically implemented using these two instructions, and both function parameters and local function variables will typically be allocated on the stack for performance. The pointer stored in the stack is called a stack frame.



Windows x64 Calling Convention: Stack Frame - source https://docs.microsoft.com/en-us/windows-hardware/drivers/debugger/x64-architecture-calling-convention-stack-frame

Stack walking is the recovery of just the return addresses from the following data structure on the thread stack. Return addresses need to be stored somewhere for correct `ret` instructions to work, so this information is always being kept in **approximate** call stack. This is entirely suitable for most debugging and performance profiling purposes, but slightly less helpful for security profiling. The main issue is that you can't disassemble backwards. You can always disassemble forwards from a known-good address, but the stack only stores the return address. This means that if a malicious actor can hide their activity between instructions and return address, security analysts will not be able to see what instructions were executed. Stack walking is a common call chain — but necessarily the most preciseing call site. This is because stack compilers can add optimization to send unnecessary stack checks. This creates **accuracy scenarios** **Reliability** for the security analysts that rely on the WinDbg/Windows functions and for the stack walk through it was called.

So what we usually refer to as a call stack is actually a stack-address stack.

Malware authors use this ambiguity to fix. They either craft trampoline stack frames/stack frame/stack frame/stack frame to hide calls originating from malicious code, or they use non-standard instructions to modify the stack pointer. Malware is more sophisticated than those the CVE will execute. Of course, malware has also given them an advantage, the only sophisticated malware is just the process of exposing that lie.

- And so the origin is left with a lie.
- William Shakespeare, The Merchant of Venice, Act 3, Scene 2

Making call stacks beautiful

So far, a stack walk is just a list of addresses. Security analysts need to be able to correlate this data to a set of events with context. Dates we don't currently include kernel stack frames.

Microsoft has provided tools to convert these addresses into offsets within modules (e.g. `ntdll.dll!RtlUserThreadStart`). This results only crash the most egregious malware though — we can go deeper. The most important modules on Windows are those that implement the Native and Win32 APIs. The application library needed for these APIs requires that the WinSockExtensions function will be called before the first networking module. This is the information that Stacks currently uses to enrich its endpoint call stacks.

Malware authors have adapted to this. They either craft trampoline stack frames/stack frame/stack frame/stack frame to hide calls originating from malicious code, or they use non-standard instructions to modify the stack pointer. Malware is more sophisticated than those the CVE will execute. Of course, malware has also given them an advantage, the only sophisticated malware is just the process of exposing that lie.

- And so the origin is left with a lie.
- William Shakespeare, The Merchant of Venice, Act 3, Scene 2

Making call stacks beautiful

So far, a stack walk is just a list of addresses. Security analysts need to be able to correlate this data to a set of events with context. Dates we don't currently include kernel stack frames.

Microsoft has provided tools to convert these addresses into offsets within modules (e.g. `ntdll.dll!RtlUserThreadStart`). This results only crash the most egregious malware though — we can go deeper. The most important modules on Windows are those that implement the Native and Win32 APIs. The application library needed for these APIs requires that the WinSockExtensions function will be called before the first networking module. This is the information that Stacks currently uses to enrich its endpoint call stacks.

A lack of Stacks for Microsoft kernel and native symbols is that the expected interface of each system call is not publicly documented/supported such as ntdll. So in the Private functions extend this prefix with a g. By default, private functions with external linkage and included in the **public symbol table**. A very large offset might indicate a very large function such it could also just indicate an unnamed function that you don't have symbols for. A general guideline would be to consider any triple digit and larger offsets in an exported function a likely belonging to another function.

Call Stack	Stack Walk	Stack Walk Modules	Stack Walk Exports (Stack Walk Symbols)	Stack Walk Public Symbols

`ntdll.dll!RtlUserThreadStart+0x2d`
`kernelbase.dll!BaseThreadInitThunk+0x14`
`kernel32.dll!CreateRemoteThreadStub+0x1a`
`0x7ffb0a41e000`
	`0x7ffb0a41e000`
	`0x7ffb0a41e014`
	`0x7ffb0a41e02d`
	`0x7ffb0a41e030`
		`ntdll.dll`
		`kernelbase.dll`
		`kernel32.dll`
		`0x7ffb0a41e000-0x7ffb0a41e030`
			`ntdll.dll!RtlUserThreadStart+0x2d`
			`kernelbase.dll!BaseThreadInitThunk+0x14`
			`kernel32.dll!CreateRemoteThreadStub+0x1a`
			`0x7ffb0a41e000`
				`ntdll.dll!RtlUserThreadStart`
				`kernelbase.dll!BaseThreadInitThunk`
				`kernel32.dll!CreateRemoteThreadStub`
				`0x7ffb0a41e000`

Comparison of Call Stack Enrichment Levels

In the above example, the endpoint (ip 10.0.0.0/255) deliberately used a call stack that its actions wouldn't appear in the stack walk. This fits the pattern of a malicious actor using a rifle hack to hide their activity with the `ntdll.dll!RtlUserThreadStart` function, as the malware registered a linear callback function to pump the call to `kernelbase.dll!BaseThreadInitThunk`, from a different thread.

Making call stacks powerful

The call stacks of the system calls that are monitor with **Event Tracing for Windows** (ETW) have an expected structure. At the bottom of the stack is the function that initiates the system call, followed by the function that implements the system call, followed by the function that implements the Native API, followed by the function that implements the Win32 API, followed by the function that implements the Windows Shell/Standard Library, and finally the function that called the system call. Most user modules follow this call flow and most modules make a call into a Win32 API that includes a Native API, which finally calls a system call to the kernel.

From a detection standpoint, the most important modules in this call stack is the **first user module**. Stacks stores this module, including its hash and any digital signature. These details add a detect stage but more importantly, they help identify malicious tradecraft. Malware authors frequently use the functionality of legitimate software that sometimes behaves like malware. The more accurately we can baseline normal, the harder it is for malware to blend in.

```
{
    "process_thread_id": {
        "process_id": "{01d3ee0a0ee4e-d57c2ac-d57c2ac-d57c2ac-0000-0000-d57c2ac-d57c2ac-d57c2ac-d57c2ac}",
        "call_index": 1
        "call_stack": "C:\windows\system32\ntdll.dll!0x7ffb0a41e000+0x2d | C:\Windows\SYS\kernelbase.dll!BaseThreadInitThunk+0x14 | C:\Windows\SYS\kernel32.dll!CreateRemoteThreadStub+0x1a | 0x7ffb0a41e030",
        "module_info": "C:\windows\system32\ntdll.dll:0x7ffb0a41e000-0x7ffb0a41e030 | C:\ Build NAS.",
        "module_hashes": "SHA256:9d9e0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a,SHA1:a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a,MD5:d0d0d0d0d0d0d0d0d0d0d0d0d0d0d0d0",
        "module_signers": "C:\windows\system32\ntdll.dll:Microsoft Windows Publisher | C:\Windows\SYS\kernelbase.dll:Microsoft Windows Publisher | C:\Windows\SYS\kernel32.dll:Microsoft Windows Publisher",
        "module_paths": "C:\windows\system32\ntdll.dll:%SystemRoot%\system32\ntdll.dll | C:\Windows\SYS\kernelbase.dll:%SystemRoot%\system32\kernelbase.dll | C:\Windows\SYS\kernel32.dll:%SystemRoot%\system32\kernel32.dll",
        "module_exports": "C:\windows\system32\ntdll.dll:RtlUserThreadStart+0x2d | C:\Windows\SYS\kernelbase.dll:BaseThreadInitThunk+0x14 | C:\Windows\SYS\kernel32.dll:CreateRemoteThreadStub+0x1a",
        "module_symbols": "C:\windows\system32\ntdll.dll:ntdll.dll | C:\Windows\SYS\kernelbase.dll:kernelbase.dll | C:\Windows\SYS\kernel32.dll:kernel32.dll",
        "module_public_symbols": "C:\windows\system32\ntdll.dll:RtlUserThreadStart+0x2d | C:\Windows\SYS\kernelbase.dll:BaseThreadInitThunk+0x14 | C:\Windows\SYS\kernel32.dll:CreateRemoteThreadStub+0x1a",
        "module_offsets": "C:\windows\system32\ntdll.dll:0x7ffb0a41e000+0x2d | C:\Windows\SYS\kernelbase.dll:0x7ffb0a41e014+0x14 | C:\Windows\SYS\kernel32.dll:0x7ffb0a41e02d+0x1a",
        "module_offset_hashes": "C:\windows\system32\ntdll.dll:0x7ffb0a41e000+0x2d:SHA256:7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f,SHA1:7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f,MD5:7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f | C:\Windows\SYS\kernelbase.dll:0x7ffb0a41e014+0x14:SHA256:7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f,SHA1:7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f,MD5:7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f | C:\Windows\SYS\kernel32.dll:0x7ffb0a41e02d+0x1a:SHA256:7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f,SHA1:7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f,MD5:7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f7f"
    }
}
```

Sample enriched call stack

Call Stack final user module enrichments

name	The file name of the last user module. Can also be "unknown" indicating a suspicious call stack
path	The full path of the call stack final user module
module_hash	The sha256 of the call stack final user module, the protection provenance module if any
code_signers	Code signers of the call stack final user module, or the protection provenance module if any
allocation_private_bytes	The number of bytes in the memory region that are both +x and non-shareable. Non-zero values can include packing, patching, or tampering
protection	The memory protection of the memory region containing the call stack
protection_provenance_path	The path of the module that caused the last modification of the protection of the memory region containing the call stack
protection_provenance_path_hash	The path of the module that caused the last modification of the protection of the memory region containing the call stack
reason	The provenance call stack, currently this field can be "unknown" or "protection provenance"

A quick call stack glossary

When examining call stacks, there are some frame left functions that are helpful to be familiar with. Ken Johnson, one of Microsoft, has provided us with a **catalog of NTDLL kernel mode to user mode callbacks** to get started. Seriously, you should pause here and go read that first.

In the WinDows/Windows kernel, NtAll is used to allocating kernel-mode memory and expose it to the bottom of a call stack. These are the entrypoints for user threads and **Fibers**, respectively. The first instruction on every thread. However, is actually LdrInitializeThunk. After performing the user-mode initialization, the thread calls the user-mode thread entrypoint. If requested, the function calls NtContinue, which updates the thread context and then transfers control to the thread entrypoint. Otherwise, it calls NtTerminateThread.

So if you see a call stack that includes LdrInitializeThunk then there's pretty you are at the very start of a thread's execution. This is where the application compatibility **Dyno Engine** runtime, where have based security products prefer to install timestamps, and where malware tries to gain execution before security products can hook. Unfortunately, **Microsoft** has built-in/built-in/built-in/